

Machine Learning

Fall 2024

Instructor: Ali Sharifi Zarchi
Homework responsible: Izadi

Sharif University of Technology
Department of Computer Engineering

Bonus deadline: December 6 — Homework 4 — Final submission deadline: December 13

- The submission deadline is 23:59 on the specified dates.
- During the semester, you may use up to 5 late days for each assignment, and at most 15 late days in total. Note that late days for the practical and theoretical parts are counted separately, and the combined delay must not exceed 15 days. After the allowed period, two extra days are provided for late uploads; during this window, 2% of the final assignment grade is deducted for every hour of delay.
- If you upload the practical or theoretical part before the bonus deadline, that part earns a 20% bonus, and afterwards a walkthrough video for the assignment will be released.
- Make sure to solve the assignments strictly according to the requirements stated in the questions. If anything is unclear, ask on the assignment page on Quera and follow the answers provided by the corresponding teaching assistant.
- If you collaborate with others or use external resources, list your collaborators and the sources you used.
- Prepare the answers to the theoretical questions as a single PDF file named HW4_T_[STD_ID].pdf, and place each practical solution in a separate zip file. Name the file for the i-th notebook HW4_P[i]_[STD_ID].zip and upload each one separately.

Assignment authors: Amirhossein Izadi, Mohammad Hossein Shalchian, Amir Ezati, Ali Bakhtiari

Theoretical Questions (100+10 pts)

1. (22 pts) Answer the following questions.

   a) Consider two consecutive layers of a deep network, such that the input layer has J feature maps and the output layer has I feature maps. The connection between these two layers can be either fully connected or convolutional. In both cases, the feature maps of the input layer have dimensions $W \times H$, and the kernel used in the convolutional layer has dimensions $K \times K$. An overview of these two layers can be seen in Figure 2.

      - If possible, compute the number of output units, connections, and learnable weights in each case; report and compare them with each other. (Ignore the bias.)

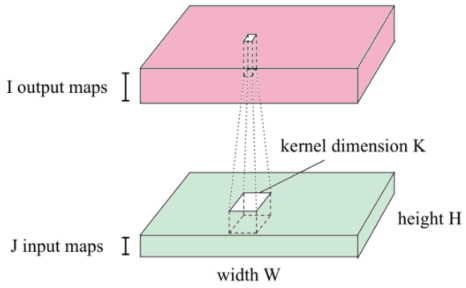


Figure 1: The input layer is shown in green and the output layer in red.

Answer:

	FC layer	**CONV layer**
# output units	WHI	WHI
# weights	W^2H^2IJ	K^2IJ
# connections	W^2H^2IJ	WHK^2IJ

(a) Can it be concluded from the previous computations that deep fully convolutional networks generally require less training data to be trained than fully connected networks? Is this conclusion necessarily correct and relevant, or not?

Answer:

- In general, deep convolutional networks require less training data than fully connected networks. The reason is that convolutional networks exploit weight sharing and a reduced number of learnable parameters. These properties make convolutional networks more efficient in their use of training data, and they need less data to reach the same level of accuracy.
- However, this conclusion is not necessarily always correct and depends on several factors. For example, if the input data is highly complex or non-local connections are important, fully connected networks may perform better. In addition, the network architecture, the amount of training data, and the way the data is processed can also have a significant effect on the data requirements.

b) Consider a CNN that uses blocks of the following form:

$$(\text{ConvLayer}) \rightarrow (\text{BatchNorm}) \rightarrow (\text{Activation})$$

- By studying this paper, compare how batch normalization is performed in fully connected networks versus convolutional networks.

Answer:

Fully connected networks: In fully connected networks, the inputs are provided as flattened one-dimensional vectors. In this structure, batch normalization is applied independently to each feature, so that the mean of the input values to each

neuron becomes zero and their variance becomes one. This reduces the variation of the input data during training and speeds up the learning process.
Also, because the statistical dependence between features is weaker in fully connected networks, normalization is performed independently. This operation stabilizes the gradients during learning and helps the model converge quickly with a higher learning rate and less sensitivity to the initial parameters.

Convolutional networks: In convolutional networks, the inputs are usually two-dimensional data or images that contain important local features. Batch normalization in these networks is designed so that all values of a feature map are normalized in the same way. This is necessary to preserve the spatial correlation of the data. For this reason, normalization is applied independently per channel and jointly across all spatial locations.
Batch normalization in convolutional networks increases gradient stability and makes learning more efficient. This property is especially important in deeper networks, because the local dependence of the data is preserved and these correlations are not disrupted.

- Does removing the bias b from the convolution layer cause any disruption to how this network functions? Why?

Answer:

- Removing the bias in layers that use batch normalization usually does not disrupt the network. The reason is that the batch normalization layer includes learnable parameters (the scale γ and the shift β) that can compensate for the absence of the bias.
- **Reason:** The affine transformation step in batch normalization (after normalization) provides the flexibility needed to compensate for the change. Therefore, the presence of the bias is not necessary.

- Also assume that the network has been trained; does multiplying the weights by a number such as α at inference time change the network's performance? What about multiplying the input values of the network by this factor?

Answer:

Effect of multiplying the weights at inference:
If we multiply the weights by α:

$$x_{\text{in}} \Rightarrow \text{Conv Layer} \Rightarrow \alpha x_{\text{out}} \Rightarrow \text{Batch Norm} \Rightarrow \alpha\gamma \left(\frac{\alpha x_{\text{out}} - \alpha\mu}{|\alpha|\sigma} \right) + \alpha\beta$$

$$= \alpha \left(\gamma \, \text{sign}(\alpha) \left(\frac{x_{\text{out}} - \mu}{\sigma} \right) + \beta \right) = \alpha y$$

Now if the activation function is non-linear, then the value $f(\alpha y)$ has no clear relation to $f(y)$, so the model's performance is affected and its behavior will not be the same as before.

Effect of multiplying the inputs at inference:
We have:

$$\alpha x_{\text{in}} \Rightarrow \text{Conv Layer} \Rightarrow \alpha x_{\text{out}} \Rightarrow \text{Batch Norm} \Rightarrow \gamma\left(\frac{\alpha x_{\text{out}} - \alpha\mu}{|\alpha|\sigma}\right) + \beta$$

$$= \text{sign}(\alpha)\left(\gamma\left(\frac{x_{\text{out}} - \mu}{\sigma}\right) + \beta\right)$$

It can be seen that if α is positive, the model's performance does not change.

- Describe an image-processing scenario in which batch normalization may perform poorly or even have the opposite effect. In such cases, what alternative techniques can be considered?

Answer:

- **Scenario:** In areas such as image enhancement, or medical image analysis, where the intensity of the input pixels has imbalanced distributions, batch normalization may lead to poor results.
  - For example, if the pixel values have a strongly imbalanced distribution, normalization may destroy fine details that are important for decision-making.

Strategies to address these problems:

- Using **Instance Normalization** instead of batch normalization in specific tasks (such as style analysis).
- Using **Group Normalization** or **Layer Normalization**, which do not depend on batch statistics.
  - Tuning the initial parameters γ and β for better adaptation to domain-specific data.
  - Re-training the network on data with a distribution similar to the test set, to properly adapt the normalization layers.

c) Consider the Unet architecture:

- Imagine that the input image dimensions for this network are 256×256. Now assume that in this architecture each encoder layer halves the dimensions and each decoder layer doubles them. At the lowest layer (the deepest layer) of this architecture, how many pixels will our feature space have?

Answer:

The input image dimensions are 256×256. In each encoder layer, the dimensions are halved. Therefore, the dimensions at each layer are as follows:

- First layer: 256×256.
- Second layer: 128×128.
- Third layer: 64×64.

- Fourth layer: 32×32.
- Fifth layer (the deepest layer): 16×16.

Therefore, at the lowest layer (the deepest layer), our feature space will have dimensions 16×16.

- Suppose the encoder has layers with $64, 128, 256$ and 512 filters. If each convolution layer uses 3×3 kernels, compute the number of parameters of the second convolution layer of the encoder.

Answer:

The number of encoder filters is $64, 128, 256, 512, 512$. For the second convolution layer with
General formula:

$$\text{parameters} = \text{output} \times \text{input} \times (3 \times 3) + \text{bias}$$

For the second convolution layer:

$$128 \times 64 \times (3 \times 3) + 128 = 73,856$$

Therefore, the number of parameters of this layer is $73,856$.

d) The interpretability of neural networks is of great importance. When classifying images, we are interested in knowing which parts of the image had the greatest influence on the classification. The first paper introduces the idea of deconvolutional networks, and the second paper introduces the idea of up-convolutional networks. By studying these papers, explain how each of the two methods leads to the interpretability of a convolutional network.

Answer:

Up-Convolutional networks: These networks, sometimes known as Transposed Convolution, are a tool for increasing the spatial dimensions of feature maps. In the paper "Inverting Visual Representations", these networks were used to reconstruct images from feature representations (SIFT, HOG, and the outputs of different AlexNet layers). In this method, an Up-Convolutional network estimates the original image by learning the inverse of the feature mapping. These reconstructions show that even in the deep layers of the network:

- the colors and overall contours of objects are preserved.
- more precise local information decreases as we move to higher layers of the network.
- high-level features such as conceptual categories (like "Tree" or "Apple") are broadly preserved.

These reconstructions help to understand how convolutional networks retain or discard information. For example, reconstruction from higher layers such as FC8 shows that even information such as colors and class prior values can be extracted.

De-Convolutional networks: In the paper "Striving for Simplicity: The All Convolutional Network", De-Convolutional networks are used as a tool for analyzing learned features. By reversing the data flow, these networks identify the image regions that specifically cause particular neurons in higher layers to activate. The process of these networks consists of the following steps:

i. Start from a specific activation in a high layer and zero out the other values.

ii. Propagate these activations back to the input image, so that the image regions with the greatest influence on that neuron are reconstructed.

iii. Use various methods for reversing the ReLU and convolution layers, such as Guided Backpropagation, which produces little noise and highlights the most influential regions.

The result of this process shows that:

- in lower layers, the activation regions are very local and related to simple patterns (such as edges).
- in higher layers, the activation regions expand to more complex and abstract patterns (such as whole objects or conceptual categories).

How this helps interpretability: Both methods contribute to the interpretability of the network in complementary ways:

- **Up-Convolutional networks:** These networks make it possible to reconstruct the original image from the feature representation. The reconstructions reveal what information each layer of the network preserves. For instance, the preservation of color and contours indicates the importance of this information in the network's decision process.
- **De-Convolutional networks:** By identifying the regions that each neuron is sensitive to, these networks help to understand the role of each feature in the network's predictions. In particular, this method shows which parts of the image have the greatest influence on the activation of higher layers.

In general, Up-Convolutional networks are suitable for reconstructing the overall image and showing the information contained in the features, whereas De-Convolutional networks focus on analyzing the critical regions that contribute to the network's decisions.

2. (8 pts) Suppose you have a vector of length N and you want to apply a one-dimensional convolution layer to it. Obtain the result of applying a convolution layer through the following relation:

$$Z = W * X \quad \longrightarrow \quad Z_i = \sum_{j=0}^{K-1} W_j X_{i+j}$$

where K denotes the filter size. If the value of $\frac{\partial L}{\partial Z_i}$ is known for all values of i, find the exact relation for $\frac{\partial L}{\partial W_j}$. Show that this relation is, in practice, equivalent to applying a convolution filter.

Answer:

A forward step of the 1D convolution operation can be written as follows:

$$Z_i = \sum_{j=0}^{K-1} W_j X_{i+j}, \quad \text{for } i \in [0, N-K]$$

To compute the gradient $\frac{\partial L}{\partial W_j}$, we can apply the chain rule:

$$\frac{\partial L}{\partial W_j} = \sum_{i=1}^{N} \frac{\partial L}{\partial Z_i} \cdot \frac{\partial Z_i}{\partial W_j}$$

We know that $\frac{\partial L}{\partial Z_i}$ is given, and we can derive $\frac{\partial Z_i}{\partial W_j}$ as follows:

$$\frac{\partial Z_i}{\partial W_j} = X_{i+j}$$

Substituting this equation into the chain rule equation, we get:

$$\frac{\partial L}{\partial W_j} = \sum_{i=1}^{N} \frac{\partial L}{\partial Z_i} \cdot X_{i+j}$$

This equation represents a convolution operation, where the gradient $\frac{\partial L}{\partial Z_i}$ is multiplied by the input X_i shifted by positions j. Therefore, the formula is in fact the result of a convolution operation. In short:

$$\frac{\partial L}{\partial W_j} = \left(\frac{\partial L}{\partial Z}\right) * X_{\text{shifted by } j}$$

or, more generally:

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial Z}\right) * X$$

That is, it suffices to convolve the upstream gradient once with the input of the convolution layer to obtain the gradient with respect to the convolution kernel.

3. (15 pts)

One of the most fundamental components of a deep network is backpropagation or Backpropagation. Consider the network given in Figure 2. In this figure, the weights of the first layer are denoted by $W^{(1)}$, the weights of the second layer by $W^{(2)}$, and the weights of the fully connected layer by $W^{(fc)}$. Also, $P^{(1)}$, $P^{(2)}$ and $P^{(3)}$ are the outputs of the first, second and third layers of the network, respectively.

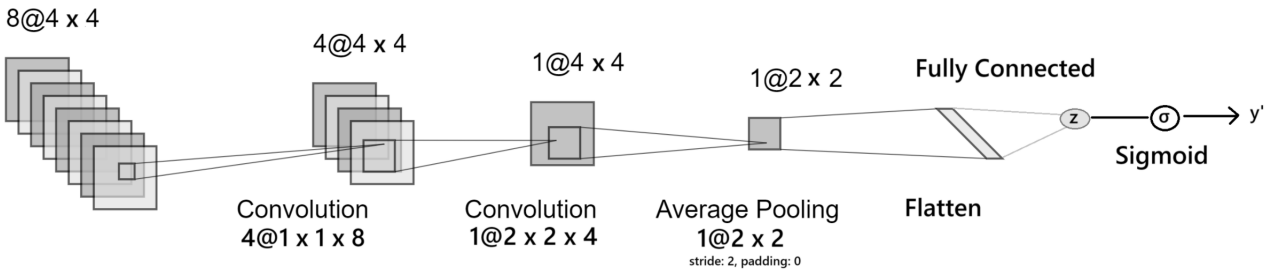


Figure 2: Network architecture

- Network input: a tensor with dimensions $4 \times 4 \times 8$
- First layer: a convolution layer with 4 filters of size $1 \times 1 \times 8$, which produces the output $P^{(1)}$ with dimensions $4 \times 4 \times 4$.
- Second layer: a convolution layer with 1 filter of size $2 \times 2 \times 4$, which produces the output $P^{(2)}$ with dimensions $4 \times 4 \times 1$.
- Third layer: an Avg. Pooling layer with a 2×2 kernel and stride 2, which produces the output $P^{(3)}$ with dimensions $2 \times 2 \times 1$.
- Fourth layer: a Flatten layer that converts $P^{(3)}$ into a vector $\mathbf{v}$ with dimensions 4×1.
- Fifth layer: a fully connected layer with a Sigmoid activation function that produces the final output.

The weights of the convolution filters in the first and second layers ($W^{(1)}$ and $W^{(2)}$) and the weights of the fully connected layer ($W^{(fc)}$) are among the trainable parameters of the network. The goal of the network is to optimize these weights to minimize a loss function L.

Tasks:

(a) Using the chain rule and based on the derivative $\frac{\partial L}{\partial z}$, compute the following expressions:

i. $\frac{\partial L}{\partial P^{(3)}_{i,j}}$: the gradient of the loss with respect to the output of the Average Pooling layer.

ii. $\frac{\partial L}{\partial P^{(1)}_{i,j,k}}$: the gradient of the loss with respect to the output of the first convolution layer.

Answer:

First, based on the given neural network architecture, let us derive the various values in the different layers.

First layer

$$P^{(1)}_{i,j,t} = \sum_{k=1}^{8} W^{(1)}_{1,1,k,t} X_{i,j,k}$$

Second layer

$$P_{i,j}^{(2)} = \sum_{k=1}^{4} \sum_{w=1}^{2} \sum_{h=1}^{2} W_{w,h,k}^{(2)} P_{i+w-1,j+h-1,k}^{(1)}$$

Third layer

$$P_{i,j}^{(3)} = \frac{1}{4} \sum_{w=2i-1}^{2i} \sum_{h=2j-1}^{2j} P_{w,h}^{(2)}$$

Fourth layer

$$h^{(4)} = P_{1,1}^{(3)}, P_{1,2}^{(3)}, P_{2,1}^{(3)}, P_{2,2}^{(3)}$$

Fifth layer

$$z = \sum_{i=1}^{4} W_i^{(\text{fc})} h_i^{(4)}$$
$$\hat{y} = \sigma(z)$$

i.

Note that we can write:

$$\frac{\partial L}{\partial h_k^{(4)}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\hat{y}}{z} \times \frac{\partial z}{\partial h_k^{(4)}} = \frac{\partial L}{\partial \hat{y}} \times \hat{y}(1-\hat{y}) \times W_k^{(\text{fc})}$$

By definition we have:

$$P_{i,j}^{(3)} = h_{2i+j-2}^{(4)} \quad (1 \leq i, j \leq 2)$$

So we obtain:

$$\frac{\partial h_k^{(4)}}{\partial P_{i,j}^{(3)}} = I(k = 2i + j - 2)$$

Consequently:

$$\frac{\partial L}{\partial P_{i,j}^{(3)}} = \sum_{k=1}^{4} \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial h_k^{(4)}} \times \frac{\partial h_k^{(4)}}{\partial P_{i,j}^{(3)}} = \frac{\partial L}{\partial \hat{y}} \times \hat{y}(1-\hat{y}) \times W_{2i+j-2}^{(\text{fc})}$$

ii.

We can write:

$$\frac{\partial L}{\partial P_{i,j,k}^{(1)}} = \sum_{m=1}^{4} \sum_{n=1}^{4} \frac{\partial L}{\partial P_{m,n}^{(2)}} \times \frac{\partial P_{m,n}^{(2)}}{\partial P_{i,j,k}^{(1)}}$$

First we have:

$$\frac{\partial L}{\partial P^{(2)}_{m,n}} = \sum_{p=1}^{2}\sum_{q=1}^{2} \frac{\partial L}{\partial P^{(3)}_{p,q}} \times \frac{\partial P^{(3)}_{p,q}}{\partial P^{(2)}_{m,n}}$$

Based on what we derived:

$$\frac{\partial P^{(3)}_{p,q}}{\partial P^{(2)}_{m,n}} = \frac{1}{4} I((m = 2p \vee m = 2p-1) \wedge (n = 2q \vee n = 2q-1))$$

$$\frac{\partial P^{(2)}_{m,n}}{\partial P^{(1)}_{i,j,k}} = W^{(2)}_{m-i,n-j,k} I((m = i \vee m = i+1) \wedge (n = j \vee n = j+1))$$

Therefore:

$$\frac{\partial L}{\partial P^{(1)}_{i,j,k}} = \frac{1}{4} \times \hat{y}(1-\hat{y}) \times \sum_{m=i}^{i+1}\sum_{n=j}^{j+1} W^{(\text{fc})}_{\lceil m/2 \rceil + \lceil n/2 \rceil - 2} W^{(2)}_{m-i,n-j,k}$$

(b) Compute the gradient of the loss with respect to the weight $W^{(1)}_{1,1,k}$, one of the filters of the first convolution layer:

$$\frac{\partial L}{\partial W^{(1)}_{1,1,k}}$$

Answer:

Now, based on the relation we derived, we have:

$$\frac{\partial L}{\partial W^{(1)}_{1,1,k}} = \frac{\partial L}{\partial P^{(1)}_{i,j,k}} \times \frac{\partial P^{(1)}_{i,j,k}}{\partial W^{(1)}_{1,1,k}} = \frac{\partial L}{\partial P^{(1)}_{i,j,k}} X_{i,j,k}$$

Substituting the previous relation:

$$\frac{\partial L}{\partial W^{(1)}_{1,1,k}} = \frac{1}{4} \times \hat{y}(1-\hat{y}) \times \sum_{m=i}^{i+1}\sum_{n=j}^{j+1} W^{(\text{fc})}_{\lceil m/2 \rceil + \lceil n/2 \rceil - 2} W^{(2)}_{m-i,n-j,k} X_{i,j,k}$$

(c) Derive the gradient expression $\frac{\partial L}{\partial W^{(2)}_{i,j,k}}$, one of the weights of the convolution filter of the second layer.

Answer:

$$\frac{\partial L}{\partial W^{(2)}_{i,j,k}} = \sum_{m=1}^{4}\sum_{n=1}^{4} \frac{\partial L}{\partial P^{(2)}_{m,n}} \times \frac{\partial P^{(2)}_{m,n}}{\partial W^{(2)}_{i,j,k}}$$

Based on what we derived earlier, we have:

$$P^{(2)}_{m,n} = \sum_{k=1}^{4}\sum_{w=1}^{2}\sum_{h=1}^{2} W^{(2)}_{w,h,k} P^{(1)}_{m+w-1,n+h-1,k}$$

Therefore, if $m+i$ and $n+j$ lie within the range of the second layer, we have:

$$\frac{\partial P^{(2)}_{m,n}}{\partial W^{(2)}_{i,j,k}} = P^{(1)}_{m+i,n+j,k}$$

and otherwise:

$$\frac{\partial P^{(2)}_{m,n}}{\partial W^{(2)}_{i,j,k}} = 0$$

Substituting, the requested derivative is computed as follows:

$$\frac{\partial L}{\partial W^{(2)}_{i,j,k}} = \frac{1}{4}\hat{y}(1-\hat{y})\sum_{m=1}^{4}\sum_{n=1}^{4} W^{(\text{fc})}_{\lceil m/2\rceil+\lceil n/2\rceil-2} P^{(1)}_{m+i-1,n+j-1,k}$$

4. (22 pts) In this question we aim to review and analyze several different variants of convolutional networks as well as CNN-based architectures introduced over the past decade.

a) DenseNet network

- State the main difference between ResNet's residual connections and DenseNet's dense connections. Also give a brief explanation of each.

Answer:

The main difference between the two models can be stated in terms of how the different layers of the network are connected, as follows:

- **DenseNet with Dense Connections:** In DenseNet, every layer is connected to all previous layers, and features are passed through concatenation. This enables feature reuse and increases the model's efficiency.
- **ResNet with Residual Connections:** In ResNet, the connection between layers is made through summation of features. This method returns gradients directly to earlier layers, but compared to DenseNet it preserves fewer features throughout the network.

DenseNet	ResNet
The output of each layer is concatenated to all subsequent layers.	The output of each layer is added to the output of the previous layer.
The gradient is transferred directly from the output layer to all layers.	The gradient returns to the earlier layers through Shortcut Connections.
Requires fewer parameters because repeated features are not considered again.	Requires more parameters due to the summation of features at each layer.
Each layer benefits from all previous features, which helps learning.	As depth increases, some layers may learn less information.
More effectively prevents the vanishing gradient problem.	Is more prone to vanishing gradient (although it is reduced).

- Explain how DenseNet reduces the vanishing gradient problem and what its computational advantage is.

Answer:

In DenseNet, each layer is directly connected to all previous layers. This direct connection allows the gradient flow from the early layers to the final layers to proceed without weakening. For this reason, the vanishing gradient problem is reduced in this architecture. The dense connectivity also improves information propagation and reduces the need for a large number of parameters.

- Considering the growth rate k in DenseNet, if each layer produces k new feature maps and the input to a dense block has 32 channels, and $k = 24$, how many output channels will the third layer in the block have?

Answer:

If the number of initial input channels is 32 and the block has 3 layers:

$$\text{number of layers} \times k + \text{initial input} = \text{number of output channels}$$

$$32 + 24 \times 3 = 32 + 72 = 104$$

$$\text{number of output channels} = 104$$

This formula shows the gradual increase of channels in each block according to the growth rate k.

b) GoogleNet network

The inception module was created with the goal of reducing computational complexity. We first get to know the inception module and then the GoogleNet network.

- In Figure 3, a convolution layer is shown in two different forms. The input in both cases is $28 \times 28 \times 192$ and the output in both is $28 \times 28 \times 32$. By computing the total number of operations performed in each case, compare the computational complexity of the two forms. Determine by what percentage the addition of the 1×1 convolution filter has decreased or increased the computational complexity.

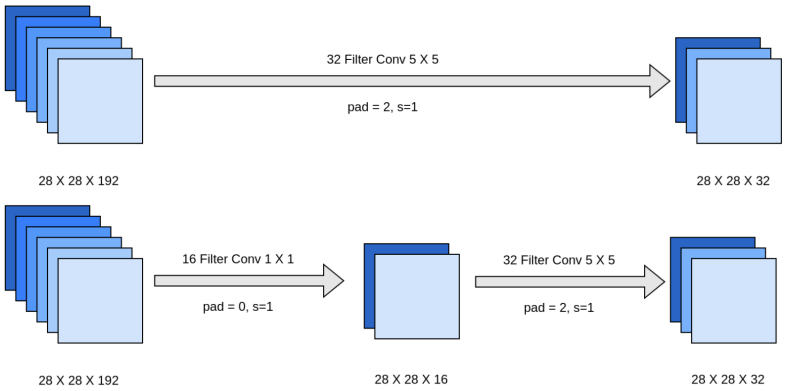


Figure 3: A standard convolution layer (top) and a convolution layer with a 1×1 filter (bottom)

Answer:

- **First case: standard convolution** (5×5)
  - In this case, 32 filters of size 5×5 are used.
  - Each filter is applied to the input $192 \times 28 \times 28$ and produces an output.
  - The total number of computational operations is computed as follows:

$$\text{operations:} \quad 28 \times 28 \times 192 \times 32 \times (5 \times 5) = 120,422,400$$

- **Second case: convolution with dimensionality reduction** (1×1)
  - In this case, the input channels are first reduced to 16 channels. Then a 5×5 convolution is applied.
  - The number of computational operations is performed in two stages:
    - **First stage: dimensionality reduction with a** (1×1) **convolution** number of operations:

$$28 \times 28 \times 192 \times 16 \times (1 \times 1) = 2,407,424$$

    - **Second stage:** 5×5 **convolution on the reduced channels** number of operations:

$$28 \times 28 \times 16 \times 32 \times (5 \times 5) = 10,648,576$$

  - **Total:**

$$2,407,424 + 10,648,576 = 130,056,000 \text{ operations}$$

- **Percentage of complexity reduction:**
  - The complexity reduction is computed with the following formula:

$$\frac{120,422,400 - 130,056,000}{120,422,400} \times 100 \approx 89.2\%$$

    It can be seen that the computational complexity of the model has decreased by roughly 90%, which is very significant.

- In Figure 4, a complete inception module is shown. The sequential composition of these modules forms the GoogleNet network. Explain the reason for using filters of different sizes (for example 1×1, 3×3 and 5×5) and even different types of filters (using Max pooling alongside convolution). Compute the total number of operations in this network.

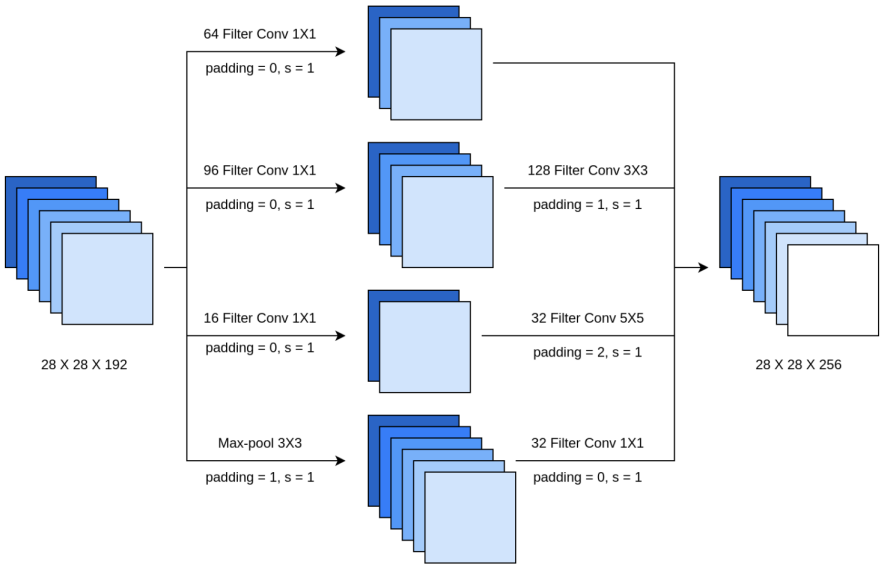


Figure 4: Overview of the inception module

Answer:

The number of operations for each type of filter in the network is computed as follows:

- For the 1×1 filter with 64 channels:
  $28 \times 28 \times 192 \times 64 \times (1 \times 1) = 9,625,600$
- For the 1×1 filter with 96 channels:
  $28 \times 28 \times 192 \times 96 \times (1 \times 1) = 14,438,400$
- For the 1×1 filter with 16 channels:
  $28 \times 28 \times 192 \times 16 \times (1 \times 1) = 2,407,424$
- For the 3×3 Max Pooling:
  $28 \times 28 \times 192 \times (3 \times 3) = 1,354,752$
- For the 3×3 filter with 128 channels:
  $28 \times 28 \times 96 \times 128 \times (3 \times 3) = 27,704,576$
- For the 5×5 filter with 32 channels:
  $28 \times 28 \times 16 \times 32 \times (5 \times 5) = 10,648,576$
- For the 1×1 filter with 32 channels:
  $28 \times 28 \times 192 \times 32 \times (1 \times 1) = 5,949,440$

Total number of operations:

$$9,625,600 + 14,438,400 + 2,407,424 + 1,354,752$$

$$+27,704,576 + 10,648,576 + 5,949,440 = 72,128,368$$

Reasons for using filters of different sizes:

- **Multi-scale processing:** The 3×3 and 5×5 filters extract larger and more complex features, while the 1×1 filter examines local features and reduces the number of channels.
- **Dimensionality reduction:** The 1×1 convolution reduces the number of input channels and optimizes the computational complexity of the larger filters.
- **Information combination:** The parallel paths extract diverse information and combine it at the output.

- Is this architecture robust against the vanishing gradient? Also explain how the auxiliary classifier in this architecture helps improve the gradient flow and the stability of optimization in the network. Finally, evaluate the possible limitations of this design in modern deep learning applications.

Answer:

GoogLeNet, introduced in 2014 as part of the Inception family of architectures, transformed the design of convolutional neural networks by focusing on efficiency, modularity, and scalability.

1. Robustness against the Vanishing Gradient problem

The vanishing gradient problem occurs when gradients decrease exponentially during backpropagation in deep networks. This problem prevents the effective training of very deep networks.
GoogLeNet addresses this problem indirectly through several architectural features:

- **Modularity in network depth:** Instead of stacking uniform convolution layers, GoogLeNet uses Inception modules. These modules are modular blocks that combine filters of different sizes in parallel. This design allows the network to be deeper without exacerbating the vanishing gradient problem, since each module focuses on extracting features at different levels.
- **Batch Normalization:** Although it was not present in the original version of GoogLeNet (it was introduced in Inception-v2), batch normalization normalizes the intermediate activations and reduces internal covariate shift, which helps gradient stability.
- **Auxiliary Classifiers:** Auxiliary classifiers are intermediate softmax layers placed at different parts of the network. These classifiers act as ”gradient boosters” and send additional gradient signals to the earlier layers during backpropagation. This feature helps reduce the vanishing gradient problem.

2. Auxiliary classifiers in GoogLeNet

Auxiliary classifiers are intermediate output layers added to help the training process. These classifiers send additional error signals to the earlier layers and improve the gradient flow.
Effectiveness:

- **Improved gradient flow:** By injecting gradient signals at intermediate points, auxiliary classifiers help reduce vanishing gradients and ensure that the early layers are trained effectively.
- **Regularization:** By forcing the intermediate layers to produce meaningful features, they stabilize training and improve generalization.

Limitations:

- **Partial solution:** These classifiers reduce the vanishing gradient problem to some extent, but they do not fully solve it, especially in very deep networks.
- **Temporary use:** The effect of these classifiers exists only during training, and they play no role in the Inference process.
- **Modern alternatives:** Techniques such as Residual Connections in ResNet have replaced auxiliary classifiers, as they provide stronger solutions for gradient flow.

Based on the above, it can be concluded that auxiliary classifiers in GoogLeNet were somewhat effective in preventing vanishing gradients, especially in networks of similar depth. However, with the emergence of superior methods in modern architectures such as residual connections, this technique has become largely obsolete.

c) Deformable convolutional networks

- Compare standard convolutional networks and Deformable convolutional networks in terms of grid sampling. Also, how can Deformable networks be more flexible with respect to Geometric transformation in images?
- What does the concept of offset mean in these networks and how is it computed?

Answer:

Standard convolution networks use a fixed sampling grid structure. For example, in a $K \times K$ kernel, the sampling points are always at predefined positions. This fixed structure gives these networks limited flexibility when facing geometric transformations.
In contrast, Deformable networks solve this problem by adding learnable offsets to the sampling grid points. These offsets, which are learned during training based on local features, allow the network to model geometric transformations dynamically and locally.
Deformable networks are able to model complex geometric transformations such as scaling, rotation, and aspect ratio changes. This property is due to the variable and learnable offsets that allow sampling from non-fixed locations.
Standard convolution equation:

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot x(p_0 + p_n)$$

Deformable convolution equation with added offsets:

$$y(p_0) = \sum_{p_n \in R} w(p_n) \cdot x(p_0 + p_n + \Delta p_n)$$

Here Δp_n are the offsets determined during the learning process.

The offsets are computed with the help of an additional convolution layer whose input is the same features of the input map. The output of this layer is an offset field at the same resolution as the input map. These offsets are learned locally and independently for each location.
For fractional locations that arise due to the offsets, bilinear interpolation is used:

$$x(p) = \sum_q G(q,p) \cdot x(q)$$

where $G(q,p)$ is the bilinear interpolation weight and p is a fractional location.
Advantages of Deformable networks:

- **Flexibility:** These networks can dynamically model complex geometric transformations.
- **High efficiency:** Due to the lightweight and simple architecture, adding these modules introduces little computational overhead.
- **Improved performance:** These networks perform better on complex tasks such as object detection and semantic segmentation.

5. (21 pts) Convolutional networks conventionally use simple convolution layers, which you are familiar with. Another type of layer that can be used in convolutional networks is the dilated (atrous) convolution layer. Figure 5 provides an intuitive illustration of the dilated convolution filter: these filters insert gaps between the cells whose information the filter uses to compute the next layer; in other words, when applying the filter and performing the convolution multiplication, we move over the input with a larger (dilated) step. Note that this step length is conceptually different from the (stride) step length in convolution layers.

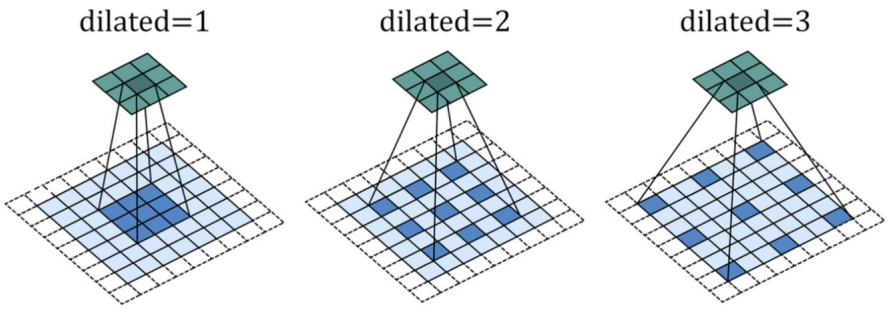


Figure 5: An illustration of dilated convolution with different dilation rates

As is also clear in Figure 5, this method is a low-cost way to increase the receptive field of convolutional networks. The dilated convolution is defined by the following closed mathematical form.

$$(K \star_D I)(i,j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} K(m,n) I(i+Dm, j+Dn)$$

a) For the input $I \in \mathbb{R}^{M \times N}$ and kernel $K \in \mathbb{R}^{F \times F}$, show that the output of the dilated convolution operator has dimensions $(M - DF + D) \times (N - DF + D)$.

Answer:

According to the assumptions of the question, the original kernel has dimensions $F \times F$. In each row and column of this kernel, the number of empty spaces is $F - 1$, and each empty space occupies size $D - 1$. Therefore, the dimensions of the kernel after dilation are computed as follows:

$$\text{New Kernel Dimensions} = F + (F - 1)(D - 1) = FD - D + 1$$

This shows that the new kernel has dimensions $(FD - D + 1) \times (FD - D + 1)$.

To compute the output dimensions of the convolution, we use the following formula:

$$\text{Output Dimensions} = (N - FD + D) \times (M - FD + D)$$

where:

$$N' = N - (FD - D) + 1 = N - FD + D$$
$$M' = M - (FD - D) + 1 = M - FD + D$$

Therefore, if the input matrix has dimensions $N \times M$, the output dimensions after applying the dilated kernel are $(N - FD + D) \times (M - FD + D)$.

b) Show that dilated convolution is equivalent to convolution with the expanded kernel K'. Specify the matrix A. ($K' = K \otimes A$, where the operator $\otimes$ is the `kronecker product`.)

Answer:

According to the question, the mathematical definition of dilated convolution is as follows:

$$(K *_D I)(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} K(m, n) \cdot I(i + Dm, j + Dn)$$

where D denotes the dilation rate and I is the input image. We also assume that the row and column numbering of the matrix starts from zero.

The Kronecker product between two matrices M and N is defined as follows:

$$(M \otimes N)_{ij} = M_{ij} \cdot N$$

This means that each element M_{ij} of matrix M is multiplied by the entire matrix N, and the result is a larger matrix. To define dilated convolution using the Kronecker product, we first define the matrix $A_{D \times D}$ as follows:

$$A_{i,j} = \begin{cases} 1 & i = j = 0 \\ 0 & \text{otherwise} \end{cases}$$

Then we compute the expanded kernel K' as follows:

$$K' = K \otimes A$$

Therefore, the elements of K' are:

$$K'_{i,j} = \begin{cases} K_{\frac{i}{D},\frac{j}{D}} & \frac{i}{D}, \frac{j}{D} \in \mathbb{Z} \\ 0 & \text{otherwise} \end{cases}$$

The output of the convolution with the expanded kernel K' is computed as follows:

$$(K' * I)(i,j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} K'(m,n) \cdot I(i+m, j+n)$$

$$= \sum_{p=-\infty}^{\infty} \sum_{q=-\infty}^{\infty} K(p,q) \cdot I(i+pD, j+qD)$$

$$= \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} K(m,n) \cdot I(i+Dm, j+Dn) = (K *_D I)(i,j)$$

which is exactly what the question asked for.

c) Suppose an input layer with dimensions $M \times N$ is fed to a neural network with three dilated convolution layers. In this network, each convolution layer contains a number of filters that slide over different parts of the input and extract its various features. Our goal is to examine the number of pixels visible to a particular element such as (i, j) of the output. In other words, find parametrically, if possible, the region of the input that influences this element.

The filter dimensions, from left to right, are:

$$(w-1) \times (w-1), \quad w \times w, \quad (w+1) \times (w+1)$$

The dilation parameter of these filters, from left to right, is:

$$d-1, \quad d, \quad d+1$$

Receptive field in convolutional neural networks

1. Receptive field in one dimension

1.1 Setup and definitions

We have L convolution layers numbered $\ell = 1, 2, \ldots, L$. For each layer ℓ, the following are defined:

- **Kernel size**: $k_\ell \in \mathbb{N}$.

- **Dilation**: $d_\ell \in \mathbb{N}$.
- **Stride**: $s_\ell \in \mathbb{N}$.

We assume there is no additional Padding. (Or we only consider the parts that actually see the input.) Two quantities are defined at layer ℓ:

- R_ℓ: the receptive field size in the original input for an output position at layer ℓ.
- jump_ℓ: the amount of movement (in units of input indices) when we move one step in the output of layer ℓ.

For the base layer $\ell = 0$:

$$R_0 = 1, \quad \text{jump}_0 = 1.$$

1.2 Layer-by-layer recurrence

For each layer $\ell \geq 1$:

$$R_\ell = R_{\ell-1} + \big(k_\ell - 1\big) \cdot d_\ell \cdot \text{jump}_{\ell-1}, \tag{1}$$

$$\text{jump}_\ell = \text{jump}_{\ell-1} \cdot s_\ell. \tag{2}$$

Intuitive explanation

- **Growth of the receptive field**: If we consider an output position at layer ℓ, that output is computed by convolving a filter of size k_ℓ and dilation d_ℓ over the output of layer $\ell - 1$. The receptive field expands by $\big(k_\ell - 1\big) \cdot d_\ell$.
- **Step size**: Moving one position in the output of layer ℓ is equivalent to moving s_ℓ positions in the output of layer $\ell - 1$.

1.3 General relations

Using the recurrence relations:

$$\begin{aligned} \text{jump}_0 &= 1, \\ \text{jump}_\ell &= \text{jump}_{\ell-1} \cdot s_\ell, \\ R_0 &= 1, \\ R_\ell &= R_{\ell-1} + \big(k_\ell - 1\big) \cdot d_\ell \cdot \text{jump}_{\ell-1}. \end{aligned}$$

2. Extension to two dimensions

2.1 Relations in the horizontal and vertical dimensions

For two-dimensional convolution, the following are defined:

- **Kernel size**: (k_ℓ^H, k_ℓ^W).
- **Dilation**: (d_ℓ^H, d_ℓ^W).
- **Stride**: (s_ℓ^H, s_ℓ^W).

The relations for each dimension $\alpha \in \{H, W\}$:

$$R_\ell^{(\alpha)} = R_{\ell-1}^{(\alpha)} + \left(k_\ell^{(\alpha)} - 1\right) \cdot d_\ell^{(\alpha)} \cdot \text{jump}_{\ell-1}^{(\alpha)},$$
$$\text{jump}_\ell^{(\alpha)} = \text{jump}_{\ell-1}^{(\alpha)} \cdot s_\ell^{(\alpha)}.$$

2.2 Square kernels with identical stride and dilation

If $k_\ell^H = k_\ell^W = k_\ell$, $d_\ell^H = d_\ell^W = d_\ell$, and $s_\ell^H = s_\ell^W = s_\ell$, the receptive field is equal in both dimensions:

$$R_\ell^{(H)} = R_\ell^{(W)} = R_\ell^{(\text{D1})}.$$

Therefore, the final two-dimensional receptive field is:

$$R_\ell^{(\text{D2})} = \left(R_\ell^{(\text{D1})}\right)^2.$$

Answer:

Computing the receptive field for the three-layer network

(a) **Layer 1**:

$$R_1 = 1 + \left(k_1 - 1\right) \cdot d_1 = 1 + (w-2)(d-1).$$

(b) **Layer 2**:

$$R_2 = R_1 + \left(k_2 - 1\right) \cdot d_2 \cdot s = \left[1 + (w-2)(d-1)\right] + (w-1)ds.$$

(c) **Layer 3**:

$$R_3 = R_2 + \left(k_3 - 1\right) \cdot d_3 \cdot s^2 \quad = \left[1 + (w-2)(d-1) + (w-1)ds\right] + w(d+1)s^2.$$

Final one-dimensional receptive field formula

$$R_3 = 1 + (w-2)(d-1) + (w-1)ds + w(d+1)s^2.$$

Final receptive field in two dimensions

For square kernels:

$$R_3^{(\text{2D})} = \left(R_3\right)^2.$$

d) Research the Masked Convolution, its applications and its limitations, and while giving a brief explanation, answer how the limitations of Masked Convolution can be improved using dilated convolution.

Answer:

Masked convolution (*Masked Convolution*) is used in deep learning, especially in models such as *PixelCNN* that are designed for sequence prediction and autoregressive models. In this method, the filters are restricted using a mask (*Mask*) so that they only use information from one or specific points. For example:

- In image processing, masked convolution ensures that the value of each pixel depends only on previous pixels.
- In sequential problems such as time-series prediction, this technique prevents access to future data.

Limitations:

- **Limited receptive field:** Masking causes each layer to access only a limited part of the input, which may be insufficient for some applications.
- **High network depth:** As the receptive field increases, a large number of layers is required, which can complicate the computations and may cause Vanishing Gradient.

Dilated convolution (Dilated Convolution) expands the receptive field without increasing the number of parameters or the number of layers, by increasing the distance between sampling points. This technique has notable advantages, especially when combined with masked convolution:

Advantages:

- **Expanding the receptive field without adding parameters:** Using dilated convolution, the receptive field can be increased significantly without adding larger kernels or more layers.
- **Preserving the local structure and preventing unauthorized access:** Combining masked convolution with dilated convolution makes it possible to preserve local regions while expanding the receptive field. The applied mask ensures that only authorized information is accessed, while dilated convolution lets the model process broader information at each layer.
- **Reducing network depth and computational complexity:** By expanding the receptive field at each layer through dilated convolution, the need for numerous layers to increase the view range is reduced. This leads to lower computational complexity and higher efficiency.
- **Increasing model flexibility:** Dilated convolution enables the model to strike an optimal balance between local precision and global information, which is essential for better understanding of different image features.

6. (22 pts) In this question we examine the vanishing gradient phenomenon.

a) (Bonus) Before the invention of methods based on convolutional neural networks, feed-forward neural networks were not able to extract features from images independently, and feature extraction was usually performed with methods other than deep learning. On the

other hand, we know that making a neural network deeper can allow the model to learn more complex features, but making the network deeper without using special techniques also introduces problems; one of the main problems that arises in this process is the vanishing gradient phenomenon.

Suppose we have a neural network with a large number of layers. We know that the gradient of the weights of layer i is computed through the following expression (where δ_L is the derivative of the loss function with respect to the output of the last layer, W_k is the weight of layer k, f is the activation function, and z is the input to the activation function).

$$\frac{\partial J}{\partial W^i} = (\delta^i)^T \times \frac{\partial z^i}{\partial W^i} = (\delta^i)^T \times a^{i-1}$$

$$\delta_i = \delta_L \left(\prod_{k=i+1}^{L} W_k \cdot f'_{k-1}(z_{k-1}) \right)$$

Assume that the largest singular value of all weight matrices W_k is smaller than one. Assuming that $\delta_L = M$ and that our activation function is of the ReLU type, show that if $L \to \infty$ then $|\delta_i| \to 0$.

Also assume that the largest singular value of all weight matrices equals 0.9. For $L = 100$ and under the stated conditions, find an upper bound for $|\delta_i|$ when $i = 0$.

Answer:

Proof that $|\delta_i| \to 0$ as $L \to \infty$

The gradient δ_i is defined as follows:

$$\delta_i = \delta_L \prod_{k=i+1}^{L} W_k \cdot f'(z_{k-1}).$$

Assuming that:

- the largest singular value of each weight matrix is $\sigma_{\max}(W_k) < 1$.
- the derivative of the ReLU activation function satisfies $\|f'(z)\| \leq 1$.

By the triangle inequality:

$$\|\delta_i\| \leq \|\delta_L\| \cdot \prod_{k=i+1}^{L} \|W_k\| \cdot \|f'(z_{k-1})\|.$$

Since $\|W_k\| = \sigma_{\max}(W_k)$ and $\sigma_{\max} < 1$, we conclude:

$$\|\delta_i\| \leq \|\delta_L\| \cdot \prod_{k=i+1}^{L} \sigma_{\max}(W_k).$$

Assuming that $\sigma_{\max}(W_k) = \sigma < 1$ for all k, we have:

$$\|\delta_i\| \leq \|\delta_L\| \cdot \sigma^{L-i}.$$

As $L \to \infty$, $\sigma^{L-i} \to 0$ because $\sigma < 1$. Therefore:

$$\|\delta_i\| \to 0.$$

Upper bound for $|\delta_i|$ when $L = 100$, $\sigma_{\max} = 0.9$ and $i = 0$

For $L = 100$ and $\sigma = 0.9$, we have:

$$\|\delta_i\| \leq \|\delta_L\| \cdot \sigma^{L-i}.$$

For example:

(a) If $i = 0$:

$$\|\delta_0\| \leq \|\delta_L\| \cdot (0.9)^{100-0} = \|\delta_L\| \cdot (0.9)^{100}.$$

Computing $(0.9)^{50}$:

$$(0.9)^{50} \approx 0.000027$$

Therefore:

$$\|\delta_{50}\| \leq \|\delta_L\| \cdot 0.000027$$

b) One approach to prevent vanishing gradient is to use a suitable initialization. Suppose we have a convolution layer with height and width 8×8 that connects a hidden layer with 3 channels to a hidden layer with 5 channels. If we use Kaiming initialization with a normal distribution, obtain the parameters of the normal distribution for initializing the kernels of this layer.

Hint: The number of input units to each kernel is the product of the number of input channels and the kernel area.

Answer:

For Kaiming Initialization, assuming the ReLU activation function, the weights of each kernel are sampled from the following normal distribution:

$$W \sim \mathcal{N}(0, \text{Var}(W)).$$

The variance of this distribution is computed as follows:

$$\text{Var}(W) = \frac{2}{n_{\text{in}}},$$

where n_{in} is the number of input units to each kernel.

Computing n_{in}:

The number of input units to each kernel is:

$$n_{\text{in}} = (\text{number of input channels}) \times (\text{kernel area}) = 3 \times (8 \times 8) = 192.$$

Normal distribution:

Substituting $n_{\text{in}} = 192$ into the variance formula:

$$\text{Var}(W) = \frac{2}{192} = 0.0104.$$

The standard deviation of the distribution is:

$$\text{Std}(W) = \sqrt{\text{Var}(W)} = \sqrt{0.0104} \approx 0.102.$$

Final result:

For each kernel, the weights are sampled from a normal distribution with the following parameters:

$$W \sim \mathcal{N}(0, 0.0104).$$

Additional note: It is also a valid answer if the standard deviation (Std $= 0.102$) is given instead of the variance (Var $= 0.0104$) among the normal distribution parameters.

c) In the early days of neural network development, the Sigmoid activation function was chosen for many networks. This function, which maps input values between zero and one, seemed suitable. But over time, as networks became deeper, it turned out that using the Sigmoid function can also cause the vanishing gradient phenomenon. One of the reasons that Modern Convolutional Neural Network architectures initially could not perform adequately was that they could not be made deep enough.

In 2012, the AlexNet architecture was introduced, which was a very significant improvement over its similar predecessor LeNet. One of the differences of this architecture was greater depth and the use of the ReLU activation function instead of Tanh / Sigmoid.

Given this, answer: why does using ReLU instead of Sigmoid help prevent the vanishing gradient problem?

Answer:

Using ReLU instead of Sigmoid helps reduce the vanishing gradient phenomenon for the following reasons:

- **The small-gradient problem in Sigmoid:** In the Sigmoid function, if the input value x moves even slightly away from 0 (whether positive or negative), the gradient becomes very small:

  $$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

  This causes the gradients to become almost zero as they propagate back to the early layers. In contrast, ReLU has a gradient equal to 1 in the range $x > 0$, which prevents this decay.

- **No saturation in ReLU:** Unlike Sigmoid, whose output is confined between 0 and 1 and leads to saturation, ReLU has no bound for positive values. This property helps the gradient not to decay in deep layers, leading to better learning.

d) Although ReLU offers many advantages, one of its drawbacks is the Dead Neurons problem; explain what this problem is. Leaky ReLU, Parametric ReLU and ELU are functions designed to address the problems of the ReLU function. Using the equation and the derivative equation of these functions, explain what advantages and disadvantages each of them has compared to the ReLU function.

Answer:

If the input of a neuron is negative, the output of ReLU becomes zero and the gradient will also be zero. This causes the neuron to no longer be updated and to remain "dead", so to speak.

Alternative functions to ReLU:

1. Leaky ReLU:

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}, \quad f'(x) = \begin{cases} 1 & x \geq 0 \\ \alpha & x < 0 \end{cases}$$

Advantages: Prevents Dead Neurons by giving a small slope ($\alpha > 0$) to negative values.
Disadvantages: Choosing a fixed value for α may not be optimal for all problems.

2. Parametric ReLU (PReLU):

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}, \quad f'(x) = \begin{cases} 1 & x \geq 0 \\ \alpha & x < 0 \end{cases}$$

α is a learnable parameter.
Advantages: Greater flexibility by learning the optimal value of α.
Disadvantages: More complexity in training and a risk of overfitting.

3. Exponential Linear Unit (ELU):

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}, \quad f'(x) = \begin{cases} 1 & x \geq 0 \\ \alpha e^x & x < 0 \end{cases}$$

Advantages: Reduces Dead Neurons and keeps the output mean close to zero, speeding up convergence.
Disadvantages: More computation due to the exponential function.

e) Consider a network with 2 residual blocks. Show how these blocks help the residual network avoid the vanishing gradient problem. Compare this network with normal networks without residual blocks. For notational simplicity, denote the function applied to the input in each block by F, which also includes the activation function.

Answer:

residual blocks help solve the vanishing gradient problem by adding the input (x) to the output of a layer ($\mathcal{F}(x)$). The mathematical structure of the two-block residual network is as follows:

$$y_1 = \mathcal{F}_1(x) + x,$$
$$y_2 = \mathcal{F}_2(y_1) + y_1.$$

To compute the gradients:

$$\frac{\partial y_2}{\partial w_2} = \mathcal{F}_2'(y_1).$$

$$\frac{\partial y_2}{\partial w_1} = (\mathcal{F}_2'(y_1) + 1) \cdot \mathcal{F}_1'(x),$$

In the normal network:

$$\frac{\partial y_2}{\partial w_2} = \mathcal{F}_2'(y_1).$$

$$\frac{\partial y_2}{\partial w_1} = \mathcal{F}_2'(y_1) \cdot \mathcal{F}_1'(x),$$

In the residual network, the presence of the extra term $+1$ in the gradient causes the gradient flow to be transferred from one block to another without severe decay. But in ordinary networks, the gradient depends only on the transformation functions $\mathcal{F}_i$, and due to successive multiplications it decays sharply as the network depth increases, which creates the vanishing gradient problem; in residual layers, however, this problem is avoided to a good extent.